DevOps Explanation Request

Perfect scenario 👍
This is **REAL-LIFE Kubernetes admin onboarding**, not theory.

I'll explain it like a story, then **hands-on commands**, and I'll be **painfully clear** about files, names, and meanings.

👑 Scenario (Real World)

- **You** = existing **Kubernetes Cluster Admin**
- **New person** = `alice`
- Alice is joining as **another administrator**
- You want:
  - Secure access
  - No passwords
  - No sharing kubeconfig
  - Proper Kubernetes-native way

👉 **Solution: TLS Client Certificates + RBAC**

☠️ Way (Wrong way – DON'T DO THIS)

- Share your kubeconfig ×
- Share admin token ×
- Disable auth ×

This is how clusters get hacked and careers end.

🛠️ Correct Way (How Kubernetes wants it)

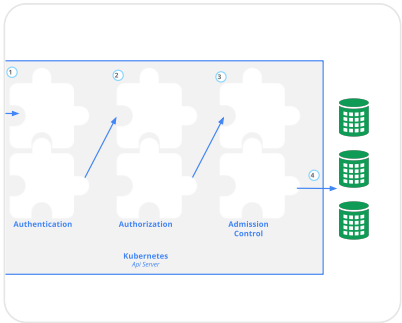
1. Create **client certificate** for Alice
2. Kubernetes **CA signs it**
3. Map Alice to **cluster-admin role**
4. Alice uses:
   - her **private key**

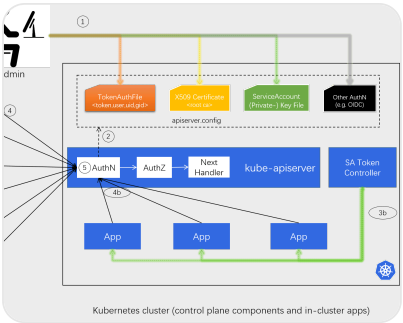
- her **certificate**
- cluster CA cert

5. Access granted securely

Step 1 – High-level Flow (Mental Model)

```pgsql
Alice → presents CERT + KEY → API Server
API Server → verifies cert with CA
RBAC → checks permissions
✅ Access granted
```





Step 2 – What YOU (Admin) Will Create

We will create access for user **alice**

Files we will end with

```text
alice.key    → PRIVATE KEY 🔐
alice.csr    → CERT REQUEST
alice.crt    → SIGNED CERTIFICATE 📜
alice.kubeconfig → login file
```

Step 3 – Generate PRIVATE KEY (Identity)

📍 Run this on **your admin machine**

```bash
openssl genrsa -out alice.key 2048
```

What this does

- Generates Alice's **private key**
- This file proves **identity**

Files created

File	Meaning
`alice.key`	🔐 PRIVATE KEY

✅ Contains `key` → PRIVATE KEY

× Never send this to anyone except Alice

Step 4 – Create CSR (Certificate Signing Request)

```bash
openssl req -new -key alice.key -out alice.csr -subj
"/CN=alice/O=system:masters"
```

VERY IMPORTANT LINE

```text
/CN=alice/O=system:masters
```

Meaning

Field	Purpose
`CN=alice`	Username in Kubernetes
`O=system:masters`	Admin group

⚠️ `system:masters` = **cluster-admin powers**

Files created

File	Meaning
`alice.csr`	CSR (request, NOT cert)

Step 5 – Encode CSR for Kubernetes

```bash
cat alice.csr | base64 | tr -d "\n"
```

- Output = base64 string
- Copy it (we'll paste into YAML)

Step 6 – Create Kubernetes CSR Object

📄 `alice-csr.yaml`

```yaml
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: alice-admin-csr
spec:
  request: <BASE64_CSR_HERE>
  signerName: kubernetes.io/kube-apiserver-client
  usages:
  - client auth
```

Apply it:

```bash
kubectl apply -f alice-csr.yaml
```

Check status:

```bash
```

```
kubectl get csr
```

➡️ Status = `Pending`

Step 7 – Approve the CSR (Admin Power)

```bash
kubectl certificate approve alice-admin-csr
```

What happens now

- Kubernetes CA signs CSR
- Alice becomes **trusted**

Step 8 – Extract SIGNED CERTIFICATE

```bash
kubectl get csr alice-admin-csr \
-o jsonpath='{.status.certificate}' \
| base64 --decode > alice.crt
```

Files created

File	Meaning
`alice.crt`	📜 SIGNED CERTIFICATE

✅ Contains `crt` → CERTIFICATE
✅ Signed by Kubernetes CA

Step 9 – Create kubeconfig for Alice

Now we bundle everything into a login file.

Get cluster details

```bash
kubectl config view --raw
```

You need:

- API server URL
- CA cert (`ca.crt`)

Create kubeconfig

```bash
kubectl config set-cluster kubernetes \
--server=https://<API_SERVER_IP>:6443 \
--certificate-authority=ca.crt \
--embed-certs=true \
--kubeconfig=alice.kubeconfig
```

Add user credentials

```bash
kubectl config set-credentials alice \
--client-certificate=alice.crt \
--client-key=alice.key \
--embed-certs=true \
--kubeconfig=alice.kubeconfig
```

Set context

```bash
kubectl config set-context alice-admin \
--cluster=kubernetes \
```

```
--user=alice \
--kubeconfig=alice.kubeconfig
```

Use context

```bash
kubectl config use-context alice-admin \
--kubeconfig=alice.kubeconfig
```

Step 10 – Hand Over to Alice 🎉

You give Alice **ONE FILE ONLY**:

```text
alice.kubeconfig
```

She runs:

```bash
export KUBECONFIG=alice.kubeconfig
kubectl get nodes
```

💥 **She is now a cluster admin**

Step 11 – How Kubernetes Auth Works Here

Item	Purpose
`alice.key`	Identity proof
`alice.crt`	Trusted by CA
`CN=alice`	Username
`O=system:masters`	Admin group

Item	Purpose
RBAC	Allows everything

Common Mistakes (Seen in Production)

× Forgetting `system:masters`
× Thinking CSR approval is automatic
× Sharing private key in chat
× Giving kubeconfig without CA
× Using tokens instead of certs for admins

🧠 Final Mental Model (Remember Forever)

```nginx
Admin creates cert → CA signs → RBAC allows
```

or simply:

> **Certificates = Authentication**
> **RBAC = Authorization**

If you want next:

- Same flow but **READ-ONLY admin**
- Removing a user (cert revocation)
- Cert expiry & rotation
- Why kubeadm clusters trust system:masters automatically

Just say it 😏🔥